



Secure and Collaborative Network Intrusion Detection:

A Federated Approach

by

Mohammed Maruf Hossen

201003812

Department of Computer Science & Engineering
School of Science, Engineering & Technology

Supervised by

Kazi Ekramul Hoque

Assistant Professor

In partial fulfillment of the requirement for the degree of *Bachelor of Science*
in *Computer Science and Engineering*

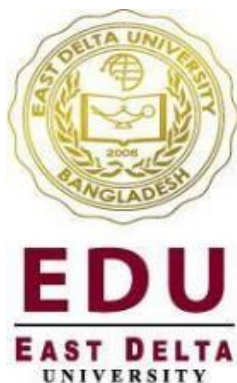
April, 2024

East Delta University

Noman Society, East Nasirabad, Chittagong-4209

Secure and Collaborative Network Intrusion Detection: A Federated Approach

This thesis is submitted in partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science & Engineering.



By

Mohammed Maruf Hossen

201003812

Supervised by

Kazi Ekramul Hoque

Assistant Professor

Department of Computer Science & Engineering

School of Science, Engineering & Technology

East Delta University, Noman Society, East Nasirabad, Chittagong-4209

The undergraduate thesis titled ‘**Secure and Collaborative Network Intrusion Detection: A Federated Approach**’ submitted by Mohammed Maruf Hossen (201003812) has been accepted as satisfactory in fulfillment of the requirement for the degree of Bachelor of Science (B.Sc.) in Computer Science and Engineering (CSE) to be awarded by East Delta University.

Board of Examiners

Dr. Mohammad Mahbubur Rahman

Chairman

Assistant Professor

School of Science, Engineering and Technology

East Delta University

Arifa Sultana Zarna

Member

Assistant Professor

School of Science, Engineering and Technology

East Delta University

Saraf Anika

Member

Lecturer

School of Science, Engineering and Technology

East Delta University

Kazi Ekramul Hoque

Supervisor

Assistant Professor

School of Science, Engineering and Technology

East Delta University

DECLARATION

It is declared that the work in this thesis is original. The information derived from the literature or work has been duly acknowledged and presented in the reference section. No part of this thesis has been submitted elsewhere for the degree, diploma, or other similar title of recognition.

Date:

Mohammed Maruf Hossen

201003812

ACKNOWLEDGMENTS

I want to acknowledge God's grace with heartfelt gratitude as he has given me the required ability, health, and strength to embark on this academic pursuit. My deepest gratitude to my supervisor is due to his never-ending support, fantastic direction, and strong motivation which set me ahead in the journey. Researching has been challenging for me as the area is complex to understand without the guidance of experts and mentors. I was lucky to have their expertise and mentorship. In addition, my heartfelt gratitude goes to my family, as well as my friends & mentors who have remained with me through my deepest times and have been drawing me with their unwavering faith and unconditional support. The care, support, and understanding of these people have been an unwavering light amid my most difficult tasks. Lastly, I express my profound gratitude to all individuals who have played a part in this academic endeavor. Their support and guidance have been instrumental in the journey towards achieving this milestone.

TABLE OF CONTENT

Contents

DECLARATION	iv
ACKNOWLEDGMENTS	v
TABLE OF CONTENT	vi
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS	x
ABSTRACT	xi
Chapter 1	1
Introduction	1
1.1 Federated Learning	2
1.2 Problem Statement.....	3
1.3 Research Goal.....	3
Chapter 2	5
Literature Review	5
Chapter 3	11
Research Methodology	11
3.2 Dataset:	13
3.3 Data Preprocessing:	16
3.3.1 Handling Missing Values:	16
3.3.2 Handling Infinite Values:	17
3.3.3 Standardization:	17
3.3.4 One-Hot Encoding:.....	18
3.3.5 SMOTE:	18
3.3.6 Isolation Forest:	19
3.4 Algorithm:	20
3.4.1 Introduction to RandomForestClassifier:	21
3.4.2 Principles of RandomForestClassifier:	21
3.4.3 How RandomForestClassifier Work:	21
3.4.4 Key Characteristics Of RandomForestClassifier:.....	21

3.5.GridSearchCV:	22
3.6 Implementation.....	22
3.6.1 Hyperparameter Tuning and Model Configuration for Edge Server 1:.....	23
3.6.2 Hyperparameter Tuning and Model Configuration for Edge Server 2.....	24
3.6.3 Hyperparameter Tuning and Model Configuration for Edge Server 3.....	25
3.6.4 Hyperparameter Tuning and Model Configuration of Edge Server 4.....	27
3.6.5 Global Model Configuration:	28
Chapter 4	31
Result and Analysis	31
4.1 Performance Evaluation:	31
4.1.1 Accuracy.....	31
4.1.2 Precision:	32
4.1.3 Recall:.....	33
4.1.4 F1-Score:	34
4.2 Result Analysis:	34
4.2.1 Model Performance:	34
Chapter 5	42
Conclusion and Future Work.....	42
References:.....	43

LIST OF TABLES

Table 3.1 Dataset Summary.....	15
Table 4.1 Test Results.....	39
Table 4.2 Detection of Edge Server 1	39
Table 4.3 Detection of Edge Server 2.....	40
Table 4.4 Detection of Edge Server 3.....	40
Table 4.5 Detection of Edge Server 4.....	40
Table 4.6 Detection of Global Server.....	41

LIST OF FIGURES

Figure 3.1 Proposed Architecture.....	12
Figure 3.2 Attack Categories.....	13
Figure 3.2 Data Distribution.....	14
Figure 4.1 Confusion Matrix of Edge Server 1 with Imbalanced Dataset.....	35
Figure 4.2 Confusion Matrix of Edge Server 1.....	36
Figure 4.3 Confusion Matrix of Edge Server 2.....	36
Figure 4.4 Confusion Matrix of Edge Server 3.....	37
Figure 4.5 Confusion Matrix of Edge Server 4.....	37
Figure 4.6 Confusion Matrix of Global Server.....	38
Figure 4.7 Model Performance.....	38

LIST OF ABBREVIATIONS

NIDS	:	Network Intrusion Detection System
IoT	:	Internet of Things
ML	:	Machine Learning
DL	:	Deep Learning
FL	:	Federated Learning
NN	:	Neural Network
CNN	:	Convolutional Neural Networks
RNNs	:	Recurrent Neural Networks
STL	:	Sparse Tensor Learning
DoS	:	Denial-of-Service
DDoS	:	Distributed Denial-of-Service
IKPDS	:	Indexed Partial Distance Search K-nearest Neighbor
KPDS	:	Partial Distance Search K-nearest Neighbor
PCA	:	Principal Component Analysis
SVM	:	Support Vector Machine
GFR	:	Gradient Feature Reduction
GBT	:	Gradient Boosting Trees
SMOTE	:	Synthetic Minority Over-sampling Technique
CICIDS2017	:	Canadian Institute for Cybersecurity Intrusion Detection System 2017
NSL-KDD	:	Network Security Laboratory- Knowledge Discovery and Data-Mining

ABSTRACT

Cyber threats keep on becoming sophisticated, making the centralized approaches for network intrusion detection encounter problems of scalability, performance, and privacy. The federated learning paradigm comes into focus which helps in the collaborative training of the model across the edge servers with data privacy preserved. This study focuses on a secure shared and collaborative network intrusion detection system using a federated approach, where a set of edge servers synchronously train a global model to avoid sharing sensitive data. A federated learning architecture is introduced, according to which edge servers perform the pre-processing and training of local models on the data stored locally. These local models are collected onto a central server to aggregate them to the global model without sharing raw data with Synthetic Minority Over-sampling Technique (SMOTE) and outlier removal as preprocessing techniques, the system shows stable performance in identifying network intrusions across a variety of edge environments. Evaluation measures such as accuracy, precision, recall, and F1-score indicate the improvement of the suggested method. In the final analysis, the federated approach for network intrusion detection is an efficient, scalability, and privacy-preserving solution that is perfectly suitable for the collaborative environment. The system employs a decentralized edge server to utilize pooled brainpower for robust security measures as well as to respect privacy laws.

Keywords: Network Intrusion Detection System (NIDS), Privacy-preserving, Federated Learning, Random Forest (RF)

Chapter 1

Introduction

Network intrusion detection is one of those crucial cybersecurity technologies that aims to protect against illegal access, malicious acts, and rising threats to computer networks. It consists of monitoring and analyzing traffic on the network to search and detect irregularities or suspect behavior that could indicate a security breach. With numerous approaches like anomaly detection [1], signature-based detection [2], and machine learning algorithms[3], network intrusion detection systems possess a fundamental value in the timely detection and prevention of cybersecurity threats, and, as a result, the security of digital environments such as networks resources is guaranteed in terms of integrity, confidentiality, and availability. In the last few years, many advanced machine learning-based approaches for network intrusion detection have been developed. Traditional machine learning-based network intrusion detection approaches face scaling up of the system, privacy-preserving, and adapting to heterogeneous network architecture.

To overcome these difficulties, Federated Learning represents an effective method used for collaboration among networks. Distributed learning process across multiple edge servers while data protection leads to solid model development without the exposure of sensitive information. The proposed architecture combines advanced machine learning algorithms and secure communication protocols which gives the privilege to edge servers to have unhindered collaboration. At the same time, will provide data safety and integrity.

1.1 Federated Learning

The decentralized machine learning system, Federated learning, as the development of a new paradigm, where collaborative model training and preservation of privacy on the edge can be achieved concurrently, has been drawing more and more attention these days [4]. Unlike the centralized approach of traditional models where the data is combined in a single place for model building, federated learning shares the training process across multiple scattered devices or servers instead of a single unit.

Briefly, federated learning enables multiple devices (e.g., smartphones, IoT gadgets, edge servers) to work together and train a collective model, without the need to share their data uselessly and risk their privacy [4]. This scheme involves distributed data collection that addresses the privacy issue of centralized data aggregation by ensuring that the user's data is kept in the devices that created it.

The federated learning process typically involves three key stages: initialization, training, and aggregation. During the initial phase of training, a chosen global model is initialized either randomly or based on the pre-trained model. Each device in the training phase uses its pocketed data to train the global model, but no data is exposed. This makes the service confidential. Lastly, in the aspect of modeling, updates of information and data from all the devices are aggregated, sometimes performed through averaging and weighted methods [5].

This interactivity style found its application in different areas such as healthcare, finance, and cybersecurity [6]. A unique advantage of network intrusion detection with federated learning is its capability to generate robust models of intrusion detection drawing from different network spaces without the need to compromise individual data privacy.

1.2 Problem Statement

A crucial part of securing computer networks from unauthorized access, malicious activities, and cyber threats is played by the Network Intrusion Detection System (NIDS). Intrusion detection can face the following challenges including:

- 1) Scalability: As the network size and complexity are growing, traditional NIDS can hardly keep track of the elevated volume and variability of network traffic. Problems with scalability may be an issue that can lead to the inability to identify and respond to cyberattacks promptly.
- 2) Adaptability: As attack types are evolving and threats are becoming more sophisticated, the Intrusion Detection System (IDS) has to also evolve to detect and respond to a range of intrusion methods that are emerging. Nevertheless, the existing systems might be insufficient and not fast enough to catch up with the always fluctuating risk world.
- 3) Privacy Concerns: Most centralized NIDS gather and process sensitive network data in a centralized point. Such a centralized approach raises data privacy and security.

1.3 Research Goal

A network intrusion detection system is the primary research goal for the code, which uses federated learning methods along with the collaborative framework to achieve the objective. This consists of introducing the federated learning technique to allow several edge servers in the vicinity of Internet of Things (IoT) devices to cooperatively train intrusion detection models without disclosing the valuable network data. Creation of custom machine learning algorithms and

communication protocols dedicated to joint model training at edge servers distributed across the network that considers the issues of communication overhead and heterogeneity of edge devices. On assessing the effectiveness and precision of the federated learning framework, with the aid of extensive experimental evaluation using real-world network datasets, performance comparison with classic centralized methods. Demonstrating solutions to the model aggregation, scalability, and privacy-reservation issues associated with federated learning in the context of network intrusion detection. Ideally, the research target is to improve network intrusion detection methods by developing a collaborative federated learning approach that ensures accuracy, scalability, and privacy while being effective in the distributed network environment in decreasing security issues.

Chapter 2

Literature Review

Network intrusion detection systems (NIDS) are no longer an option but a necessity of the modern IT security ecosystem. Because cyber attackers tend to be more intelligent and their strategies more intricate, the need for reliable firewalls and intrusion detection methods is growing.

The literature review intends to present a thorough outline of the latest research related to network intrusion detection techniques, methods, and improvements.

Various machine-learning algorithms have been used for NIDS over the decade. A high-performance NIDS was conducted by R. Sekar et al. (1999) [7]. They applied a domain-specific-language approach to normal and/or abnormal network packet sequences, the system is based on an efficient algorithm that uses pattern matching and information aggregation on the sequence of network packets which had a remarkable performance for detecting attacks at that time.

As the attacks are evolving and becoming more sophisticated, the ways of detecting them are also getting advanced. Q. Niyaz et al. (2015) demonstrate recent progress in applying deep learning for cybersecurity risk detection covering network intrusion detection based on different deep learning architectures, for example, Convolutional Neural Networks (CNNs), and Recurrent Neural Networks RNNs[8]. The combination enhances the ability to identify cyber threats Q.Niyaz et al (2015) used Sparse Tensor Learning (STL), a deep learning-based technique for building the NIDS, as we know Deep Learning (DL) has its advantages such as learning features from raw data and has the potential to handle large-scale and complex datasets.

A Random Forest based NIDS was introduced by N. Farnaaz and M. A. Jabbar (2016) which covered existing research that has proven Random Forest (RF) based protocols being effective in the detection of different kinds of network attacks, for instance, denial-of-service (DoS) attacks, distributed denial-of-service (DDoS), intrusion attempts, and other malicious activities[9]. The benefits of using RF compared with other learning machines for NIDS classification also RF's capability to handle high-dimension data, and its capability to endure noisy/ non-structured data. Its ability to break down complex relationships between features and its scalability to huge sets make it an efficient machine-learning algorithm for NIDS with higher accuracy than the other models.

Aiming at the speed of the k-nearest neighbor (kNN) classifiers B. Brao and K. Swathi proposed two fast kNN algorithms based on Indexed Partial Distance Search kNearest Neighbor (IKPDS) and Partial Distance Search kNearest Neighbor (KPDS) algorithms and compared them with the traditional kNN classification. Employing the Network Security Laboratory- Knowledge Discovery and Data-Mining (NSL-KDD) dataset, the authors used 10-fold cross-validation to evaluate the performance of their algorithms[10]. The IKPDS algorithm especially improved the classification time by a large value. The paper gives an impeccable description of the fast kNN classifiers adoption in NIDS. The proposed algorithms significantly decrease the time taken for classification without decreasing accuracy. This represents a prominent advancement in the network security field.

Perez et al. (2017) implementation is significant for the area of the intrusion detection field by providing an overview of the use of hybrid Machine Learning (ML) techniques for the improvement of security measures in situations where cyber threats are progressing. It deals with the fundamental security issues in the cloud computing paradigms and the Internet of Things (IoT)

environment[11]. The authors state that hybrid ML methodology can be used to automate the intrusion detection process which is a critical issue in the context of the dynamic nature of cyber threats. The study is concerned with the development, execution, and performance assessment of numerous hybrid ML models. These models merge supervised and unsupervised algorithms to construct a resilient system that can discover not only the known attacks but also the new attacks, thereby bypassing the drawback of traditional security systems that depend on predefined rules. The outcomes reveal that merging different approaches of ML improves the system's adaptability and decreases the number of false alarms.

The work of A. Pattawaro and C. Polprasert in intrusion detection networks also provide a method that involves feature selection, clustering, and classification procedures for the system of the network. They suggested a system that combines feature selection techniques such as Principal Component Analysis (PCA), and Gradient Feature Reduction (GFR) along with hybrid machine learning algorithms using Support Vector Machine (SVM), K-means clustering, and Neural Network (NN)[12]. The system performance is critically checked by using the NSL-KDD dataset and the KDDTest+ dataset. The research puts forward an issue of a highly sturdy network security system and proposes a new solution that applies a combination of a few machine learning methods. The authors have a detailed critical analysis of the currently available knowledge within the area of professional communication, with a specific orientation toward network intrusion detection.

The contribution of O. Faker and E. Dogdu (2019) makes a breakthrough in the area of intrusion detection systems, which connects the methods of ML and DL[13]. This study has put forth a taxonomy that categorizes intrusion detection systems using the unit of data objects, which is essential in the creation of an extensive understanding of machine learning-based and deep learning-based techniques. DL methods do this by themselves and put forward those subtle but

crucial differences as well as achieving high accuracy. O. Faker et al. proposed method gets its start with the preprocessing of datasets and then proceeds with feature selection. Here the K-means clustering is used with homogeneity metric as an unsupervised feature selection technique which helps in improving the performance of classifiers. Machine learning models are evaluated using a five-fold cross-validation technique to ensure their proper performance. A deep neural network and two ensemble methods (RF and GBT) are applied to learn the models reducing the dimensionality to subsets of relevant features.

“Intrusion Detection Systems using Supervised Machine Learning Techniques: The Study of “A Survey” by E. E. Abdallah et al(2022), gives a comprehensive knowledge of intrusion detection systems (IDS) that incorporate supervised machine learning (ML) methods. The study aims to establish a taxonomy for IDS and supervised ML algorithms, offering a deep discussion on the concepts of IDS, supervised learning techniques, and cybersecurity attacks highlighting the high classification performance of supervised learning algorithms based on a study of four popular datasets in the domain: KDD’99, NSL-KDD, CICIDS2017, and UNSW-NB15[14]. It additionally emphasizes feature selection which is sometimes necessary to boost model performance. Additionally, the paper tackles the problem of data imbalance and posits that sampling techniques have the potential to resolve it. In their survey, Abdallah et al. power supervised learning across intrusion detection efforts as part of their taxonomy. This hierarchy entails the finding that supervised learning algorithms can be promising in the domain of IDS because of their capacity to detect known attacks with high precision and low false alarms.

These centralized data processing methods potentially threaten privacy, and are additionally often inefficient to handle the huge volume of data generated from IoT devices. Rey et al.(2022) rely on the N-BaIoT dataset which is furthermore used to limit real-world network traffic from different

infected IoT devices to assess the adoption of the proposed federated learning framework. They designed both supervised and unsupervised learning models, namely a multi-layer perceptron and an autoencoder for recognizing malware in both known (seen) and unknown (unseen) IoT devices. The performance of these federated models is compared with traditional approaches: one where each participant separately trains a model with their data or another way where the participants collaborate by sharing their data with a central entity that trains a model globally. The investigation reveals that federated and centralized approaches, which rely on diverse, extensive data pools, are the most effective when it comes to model performance. Interestingly, the Model gives the same output as the Centralized one even while the privacy of participants' data is protected[15]. Achieving this outcome highlights that federated learning can be a comprehensible solution for IoT security problems. Nevertheless, the study demonstrates the exposure of federated learning to adversarial attacks. The most widely used federated learning aggregation technique - is however vulnerable to that kind of attack, even a single corrupted node. The authors look at alternative model aggregation functions as compensating mechanisms, showing better onset of these attacks. The paper concludes that although remarkable progress has been made in terms of federated learning, more research work is needed to fortify the theorem proofs against adversarial attacks.

In resource-scarce situations where conventional data processing through central methods is not viable. The study of Y. Qin and M. Kondo (2021) provided a ground for more investigations in federated learning applications for intrusion detection systems[16]. It is based on a new technique that uses federated learning (FL) and feature selection methods to detect network intrusion. The system proposed by the authors is designed to tackle the complicated and varied threats of network attacks using a distributed machine-learning platform. This framework enables users to train their predictive models collaboratively while all privacy-sensitive information remains confidential.

The main contribution of the paper is the development of a greedy algorithm for feature selection that works to increase intrusion detection precision for diverse attack types. Using the principal characteristics in traffic network data, the system can better detect possible threats. Next, the authors apply a federated learning technique for creating multiple global models based on the features chosen by local devices.

The contribution of J. Li et al. (2023) to the problem of network intrusion detection in the federated learning application gains much practical value[17]. It argues for a well-defined mechanism addressing the major issues of efficiency, privacy, and scalability, that will be useful for researchers and developers in the network security context. The authors evaluate the system extensively and have done experimental demonstrations which have shown the system to be effective in detecting network intrusions of both minor and more serious varieties. Comparative results show that the proposed federated learning system fulfills the given detection accuracy target and drastically minimizes communication costs compared to centralized approaches. The authors additionally examine the problem of communication overhead, which is one of the Federated learning systems' bottleneck problems. These developers come up with optimized methodologies that cut down the overall volume of unnecessary data, therefore, the system is scalable and efficient.

Chapter 3

Research Methodology

3.1 System Architecture of Proposed Model:

The suggested model focuses on the cooperative training of a system-wide spanning net of distributed edge servers that are each characterized by an individual dataset. The architecture consists of a set of interconnected components that are designed to perform fast data processing, model training, aggregation, and evaluation. The core of the system is the preprocessing component that takes care of the data cleaning, standardization, and encoding of the raw data from each edge server's datastore. In the preprocessing section, data uniformity and compatibility are corrected among different servers in such a way that missing values and inconsistencies are handled. In this step the data preprocessing is carried on, later the model training runs internally in each edge server. Random Forest classifiers are trained by a federated learning method tailored to a specific data set features of each server on the cloud. Furthermore, the applied method includes the Isolation Forest for outlier detection and SMOTE for data balancing to ensure model robustness and performance. Once a local model is trained, the learned parameters or features are transmitted to a central aggregator, which is where model aggregation takes place. Once trained, the local models transmit their parameters to a central aggregator, where model aggregation takes place. The aggregation process typically involves weighted averaging of model parameters, accounting for variations in dataset sizes and distributions across edge servers. It goes without saying that after distributed information is utilized in a global model, such a model represents a compound knowledge of data drawn from every single available dataset under the protection of data privacy

and security. Eventually, the global model gets evaluated with a set of test data passed on to a centralized server.

Briefly, the paradigm creates a decentralized network of components and safeguards the confidentiality of distributed edge servers via the federated learning principle. Using data preprocessing, local model training, aggregation, and performance evaluation as being one entity in the architecture, the system simplifies the transfer of knowledge, development of models and ensures that sensitive data is protected and for real-life performance.

The entire categorization procedure is illustrated in Figure 3.1.

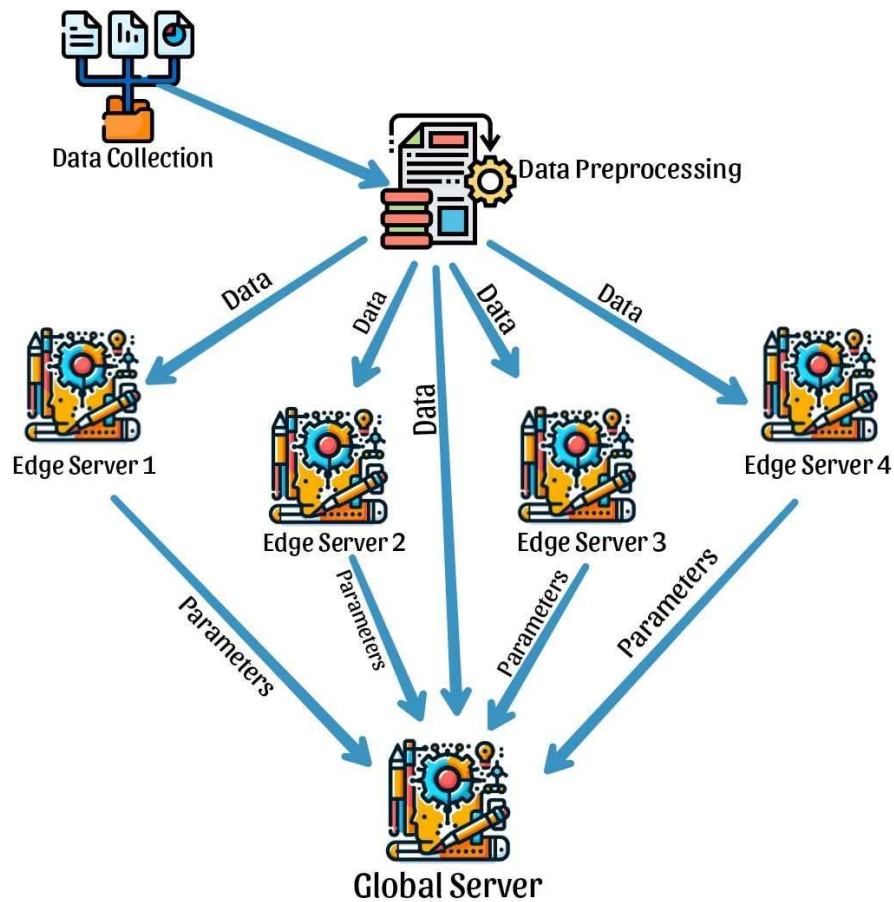


Figure 3.1 Proposed Architecture

3.2 Dataset:

I went for the Canadian Institute for Cybersecurity Intrusion Detection Systems 2017 dataset (CICIDS2017) [18] which is one of the benchmark datasets commonly used in network intrusion detection systems. It is network traffic data collected in a realistic network environment, including both honest and other hostile activities. Featuring more than 80 traffic network features including packet sizes, protocol types, and connection duration, the dataset captures all the network behavior in detail. Every data record has labels with a known set of attack categories, for example, DoS, IDS attacks, or scanning ports, that are used for training and testing of intrusion detection models. Before model training, the dataset was preprocessed to handle the presence of missing values, apply the scaling of features and balance class distribution. Apart from that, to reject biased model evaluation, the datasets were split into training and testing subsets, providing diverse training data sets and appropriate test data for model testing on unseen traffic patterns. Figure 3.2 illustrates the target class ‘Label’ which we will try to detect in the dataset.

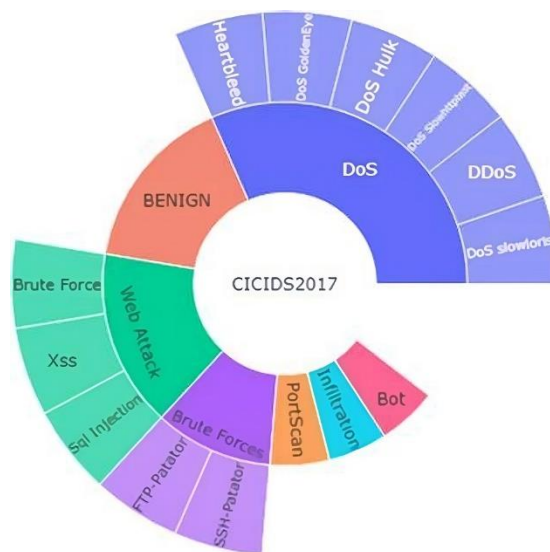


Figure 3.2 Attack Categories

For data distribution, each edge server receives approximately 20% of the total dataset for training. So, the data distribution is balanced among the edge servers, with each receiving an equal share of the dataset. With the allocated dataset to every server, the servers are then split into train test ratios for training and testing. Figure 3.3 shows the data distribution among the servers.

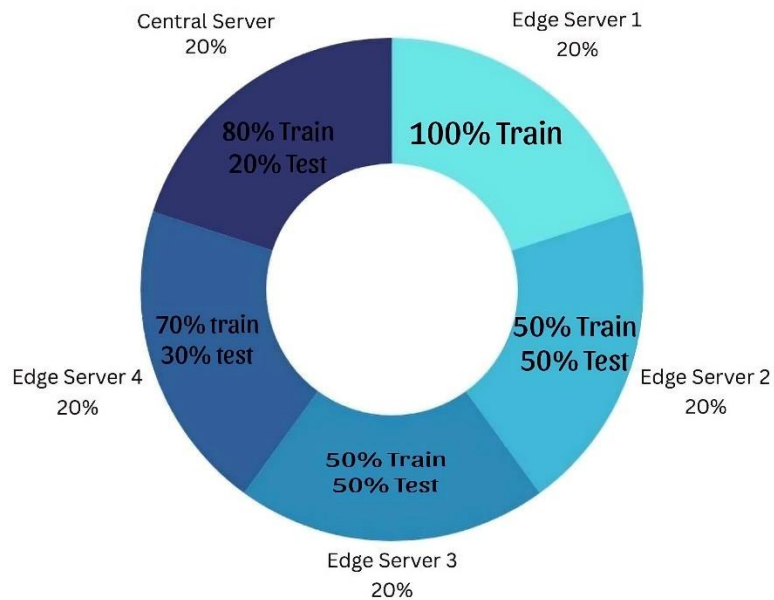


Figure 3.3 Data Distribution.

Edge Server 1: 100% Train Data

Edge Server 2: 50% Train and 50% Test

Edge Server 3: 50% Train and 50% Test

Edge Server 4: 70% Train and 30% Test

Central Server: 80% Train and 20% Test

Table 3.1 describes the type of data in the dataset.

Table 3.1 Dataset Summary

Feature Name	Description
Source IP Address	The IP address of the source (sender) of the network traffic
Destination IP Address	The IP address of the destination (receiver) of the network traffic
Protocol	The network protocol used (e.g., TCP, UDP)
Source Port	The port number used by the source in the network communication
Destination Port	The port number used by the destination in the network communication
Bytes Transferred	The number of bytes transferred in the network communication.
Packets Transferred	The number of packets transferred in the network communication
Duration	The duration of the network communication
Flag	Flags associated with the network communication (e.g., SYN, ACK)
Label	The label indicating whether the network traffic is benign or malicious (e.g., Normal, Botnet, DoS)

3.3 Data Preprocessing:

Aiming at applying the federated approach to the NIDS implementation, I employ edge servers applicable to their data to run machine learning algorithms to identify network intrusions formed in the area of their confluence. This process is decentralized which implies that sensitive information can't pass any borders of individual servers. In turn, they remain safe from data breaches and unauthorized access. Besides, the clearing of every edge server is of significant value in the right working of intrusion detection systems. After that, the preprocessing pipeline accomplishes the task of normalizing or removing the missing values, handling the outliers, and balancing the class distribution. In this way, all the edge servers get the consistent input data. So finally this supports elaboration on involvement in the generation of accurate and reliable intrusion detection models that complement the specificities of the particular network segment.

3.3.1 Handling Missing Values:

Dealing with the null values was carefully managed using the thorough and standard preprocessing pipeline. Missing values, that are not dealt with properly, have serious negative consequences for the credibility of machine learning algorithms. Therefore, the treatment of the dataset with SimpleImputer preceded the data augmentation. This module dealt with missing data efficiently; it utilized techniques like mean, median, or most frequent value imputation for each column. With this approach, the valuable data was in the meantime salvaged and the overall coherence and unity of the dataset were held. As a result of this, subsequent analyses and model training occurred based on the completed and representative set of features. Finally, the existing

Missing Value Systematically Addressed helped to fortify Dataset, thus creating a solid basis for model building.

3.3.2 Handling Infinite Values:

To prevent the infinity values troubles within the dataset, a high standard of preprocessing protocol was conducted as the first step. In the case of infinite functions, the absence of human understanding could lead to an increase in inaccuracies and instability of the model, which over time could degrade the effectiveness of the system. However, this dataset still carries the risk that will be mitigated using the replace function. The latter method reaches an end where infinite value wholly gets replaced by NaN (not a number) and therefore rids the process of further analysis and model training of the consequences of infinity. When the NaN (not calculated) values were indicated, they were at first replaced and adequate imputation techniques, such as the mean or median imputation, were done to still preserve the valuable data and at the same time bring about the restoration of the dataset's integrity. In a systematic approach of assigning syntactic values, the preprocessing had prepped the dataset to a sturdy foundation that implies good error handling while doing subsequent modeling and enhancing the reliability of the analytical process.

3.3.3 Standardization:

The standardization process was carried out with great care to eliminate skewness in the scale or distribution of the data which could affect the performance of the model. Standardization represents a fundamental preprocessing stage that gives the task of uniformizing the numerical features and hence raises the precision and stability of machine learning models. Utilizing the MeanScaler, this transformation makes sure that the mean of all numerical features is 0 and the

standard deviation is 1. This crucial step reduces the impact of outliers, and inconsistencies in feature scale units, which in turn helps in the formation of a more uniform and harmonized representation of the data set. The standardization of the numerical features by the preprocessing pipeline promotes an environment in which the model training is satisfyingly precise and quick leading to a model that generalizes very well even to the unseen data.

3.3.4 One-Hot Encoding:

To make efficient and effective preprocessing as a whole, the dataset has the feature of meticulous encoding to handle categorical variables so that the model could become capable of interpreting and extracting needed information in a meaningful way from data. One such utility technique that was in use was one-hot encoding, an approach fashioned to transform categorical variables into a numerical format that machine learning algorithms could use. A OneHotEncoder was used which dichotomized each feature by designing a binary representation for each category with one distinct column for each category. The fact that this process allows the correct assigning of categorical variables to their respective classes without adding any implicit ordering makes the data integrity intact. Through the encoding process and categorical variables transformations into a format favorable for model interpretation, the preprocessing pipeline gives machine learning models the ability to use these categorical variables more effectively in their predictive undertakings, thus adding strength to these models and the analytical process.

3.3.5 SMOTE:

To fortify the dataset against class imbalances that could potentially hinder model training and performance, a strategic preprocessing technique known as Synthetic Minority Over-sampling

Technique (SMOTE) was employed. SMOTE is a pivotal step in addressing the challenge of class imbalance, particularly prevalent in datasets where certain classes are significantly underrepresented compared to others. By synthetically generating new instances of the minority class, SMOTE effectively rebalances the class distribution, ensuring that each class is adequately represented in the training data. This process not only mitigates the risk of model bias towards the majority class but also enhances the model's ability to learn discriminative features from the minority class. Through the application of SMOTE, the preprocessing pipeline optimizes the dataset for more robust and accurate model training, ultimately leading to improved predictive performance and generalization on unseen data.

3.3.6 Isolation Forest:

To prevent these anomalies and outliers from impairing the model performance, an Isolation Forest algorithm was used as a robust preprocessing tool. Isolation Forest is a promising technique that stands out in the area of outlier detection, by building binary trees and separating them faster than many other methods. Isolation Forest's ability to locate irregularities in the dataset is skillfully done by dividing the typical values from the anomalous ones. Through this proactive method not only is the deleterious impact of outliers neutralized but also subsequent machine learning algorithms are made more robust and fault-tolerant as they are trained on the purged and more reliable dataset.

$$S(x, n) = 2^{-\frac{E(h(n))}{c(n)}} \quad (1)$$

$h(n)$ is the path length of instance x in the tree, $E(h(n))$ is the average path length of an unsuccessful search in a binary tree with n normal instances, and $c(n)$ is the average path length of successful search in a binary tree with instances. In the end, the preprocessing management turns out to be much better at detecting the patterns and relationships in the raw data, thus improving the model's potential to make correct predictions for the unseen data too.

3.4 Algorithm:

In the study, I used a Random Forest Classifier algorithm for a remote and a central server classification case scenario. RandomForestClassifier machine learning method creates a range of decision trees and classifies the final model (classification) or takes the average (regression) of individual trees. It is a versatile algorithm that has a reputation for its reliability and intricate data-handling traits.

My implementation had RandomForestClassifier models trained on each edge server taking advantage of local data to minimize the available data. The optimization of hyperparameter tuning was carried out by using GridSearchCV to train the model. The parameters of great importance were the number of estimators, the maximum depth of the trees, and the minimum number of samples required to move the node. Additionally, a feature importance analysis was conducted to reveal the most significant features that contributed to the classification goal. This examination led me to comprehend the significance of varied attributes in prediction making and ultimately, gain knowledge on the characteristics of the dataset. After training local models their consecutive parameters, which include feature importance and hyperparameters, were aggregated at the central server to fabricate the global model. This global-scale model was meant to merge all edge servers' knowledge whilst keeping individual privacy and security safe.

3.4.1 Introduction to RandomForestClassifier:

Random Forest is a strong ensemble learning method employed in machine learning for solving classification problems. It works by building up many decision trees during training and outputs the class which is the mode of the clusters of the individual trees. Random Forest is known for its strength, scalability, and capacity to process high-dimensional data that contain complicated relationships, so, it is a popular option among many classification tasks in different domains.

3.4.2 Principles of RandomForestClassifier:

The RandomForestClassifier stands out by generating several decision trees during training. Each tree is randomly assembled from a feature subsample and it employs a random subset of the training data. This randomization therefore serves as the source of diversity by which the trees' functions get improved regionalization.

3.4.3 How RandomForestClassifier Work:

Trees of each forest independently classify the data and the aggregation of the individual trees prediction determines the final result. The modal class of the class predictions across all trees is taken as the final prediction when the classifications are set.

3.4.4 Key Characteristics Of RandomForestClassifier:

RandomForestClassifier as an algorithm can carry out classification as well as regression jobs[19], therefore, it is suitable for a wide range of machine-learning problems. Through the combination of the ensemble nature of the random forest classifier, the classifier is resistant to noisy data and outliers. RandomForestClassifier produces a set of feature importance objective functions, which helps to understand how much each feature is contributing to the overall classification process.

3.5.GridSearchCV:

GridSearchCV is a powerful technique in machine learning that can be applied for hyperparameter tuning to discover the optimum set of hyperparameters. Using the grid search approach it conducts a comprehensive electronic search in this subset of hyperparameters and their possible values, evaluating each combination by using cross-validation to determine the model's test results[20]. The grid search algorithm evaluates every possible combination of hyperparameters that are chosen and formed into a grid, each single point in the grid represents a parameter's value.

$$Grid\ Search = Cartesian\ Product\ (P_1, P_2, \dots, P_n) \quad (2)$$

(P_i) represents the set of possible values for the i^{th} hyperparameter. The Cartesian product generates all possible combinations of hyperparameter values. By evaluating each combination, GridSearchCV helps in finding the hyperparameter values that lead to the best performance of the model.

3.6 Implementation

To build a secure and collaborative NIDS I selected CICIDS2017 Dataset and applied data preprocessing. To perform distributed machine learning where, unlike the traditional machine learning techniques, then distributed the data equally within the servers. As the robust and clean

data were distributed to the servers trained the edge servers based on their allocated dataset.

Within the servers, the allocated data are again split to train data and test data for the detection system model.

3.6.1 Hyperparameter Tuning and Model Configuration for Edge Server 1:

In the federated learning architecture, server-specific hyperparameter configurations fulfilling the needs of edge devices are vital to getting the model to perform optimally. Hyperparameters fine-tuning using Edge Server 1 was performed cautiously to accommodate the specificity of the model parameters. The process required performing an exhaustive search for every value of hyperparameter over the previously defined hyperparameter grid which was performed by the GridSearchCV method, aiming to find out the best set of hyperparameters for the RandomForestClassifier algorithm. The six major adjustable variables included the number of estimators (trees), the maximum depth of the tree, and the minimum number of samples split. These parameters were however carefully calibrated to allow for a trade-off between model complexity and generalization capability, also bearing in mind limited computing resources and possible data distribution variations at Edge Server 1.

The hyperparameter tuning process involved investigating a grid of the specific hyperparameter values that were specific to a given problem and constraint. For example, the number of estimators can run from 50 to 200, tree depth can be tested in the range of 5 to 20, and split points can be estimated from 2, 5, and 10.

Furthermore, the best settings for the RandomForestClassifier model at the Edge Server 1 were identified after going through hyperparameter tuning tests. This configuration guaranteed that the

model was to be highly refined to match the local data characteristics and computational capabilities, which in turn improved its ability to forecast into the network.

Moreover, a vital issue was the extraction of the standard hyperparameter values and model settings after the model tuning phase. The extracted parameters made it possible to know the final settings of the RandomForestClassifier model at Edge Server 1, which then helped to bring up the reproducibility and transparency in the federated learning platform. For better implementation and impact of the servers certain weights are assigned to them based on their performance criteria, this assigned weight will determine which servers will impact more in the development of the global server.

3.6.2 Hyperparameter Tuning and Model Configuration for Edge Server 2

For Edge Server 2, which makes up the other crucial subsystem of the distributed system, I pursued a thorough approach towards fine-tuning hyperparameters and optimizing model parameters. GridSearchCV was used to evaluate the complete list of predefined hyperparameters and discover an optimal configuration of the RandomForestClassifier algorithm. Parameters including the number of trees (estimators), tree depth, as well as the minimum number of samples for splitting were well chosen. This cyclic operation was sought to ensure that the model would have the ability to make accurate predictions, given the constraints of computation and variable data distribution at Server 2. The hyperparameter tuning phase indeed encompasses a wide spectrum of values distinctive to the problem domain and the available computing resources as well. One example is when the number of estimators was pushed from 50 to 200, tree depths were between 10 and 30, and the splitting sample size was set at 2, 4, and 8.

The model tuning process and decided upon the best model configuration for the RandomForestClassifier at Edge Server 2. Such a design was deliberately made to correspond with Local Data attributes and Edge Server 2's computing power constraints, which allows the model to work efficiently with the distributed setup.

Furthermore, I specified a specific weight for Edge Server 2 as compared to others which are pertinent in the federated system. This weight, computed by taking into account data set size and computational capabilities, was designed to appropriately measure Edge Server 2's influence during the model aggregation process to give a reasonable account of its input as well as the network's collective intelligence.

Moreover, along with choosing the best hyperparameter values and model configuration, I bring to light the final settings of the RandomForestClassifier model at the Edge Server 2 which has been tuned into the best model with weight assigned like the Edge Server 1. Those extracted parameters and weights improved in terms of transparency and reproducibility by the federated learning framework, while this fact strengthened its reliability and effectiveness.

3.6.3 Hyperparameter Tuning and Model Configuration for Edge Server 3

With Edge Server 3 of the distributed system that is an individual segment, except for parameters and hyperparameters and a model for fine-tuning hyperparameters and parameters optimization, I follow a systematic approach. By the GridSearchCV method, I have conducted a deep experiment about all hyperparameters in this predefined grid to find the optimal combination of the RandomForestClassifier algorithm. The necessary parameters such as the number of estimators, the maximum depth of the tree, and a minimum sample for splitting were adjusted in detail. Therefore, the cyclic approach aimed to tackle two key difficulties – concerning model complexity

and data model generalization ability on one hand and Edge Server 3 capabilities and potential data variations on another hand.

Server 3's Edge hyperparameter tuning phase had a tailored tuning process devoted to the domain problem and the available computing tools. Using a radial basis function, for example, I studied the impact of increasing the number of estimators (50, 100, and 200), changing tree depths (5, 10, and 20), and applying different splits (3, 6, and 10).

Moreover, the same data were divided into training and testing to be fit for Edge Server 3. Data allocation ratio (that is the several training-related measures as well as the generalization of the model being assessed) — one of the most important model performance indicators. Edge Server 3 was optimized to harness a specific syntax whereby training and testing are performed in a ratio of a specific value so that neither training nor testing on unseen data is compromised excessively. After fine-tuning the hyperparameters and optimization for data splitting, I make a direct evaluation for RandomForestClassifier model with Edge Server 3 as the best configuration. This rudimentary configuration was meticulously put together based on the Edge Server 3 unique set of data characteristics and computation constraints, and hence, it exhibited the best-ever model performance in a distributed ecosystem.

On the other hand, together with other edged servers, the Edge Server 3 also was assigned a weight coefficient that represented the node's significance among the other nodes in the system. The assigned weights based on dataset factors and computational capabilities determine the impact of Server 3 in the global model creation.

3.6.4 Hyperparameter Tuning and Model Configuration of Edge Server 4

Likewise, I carefully did the tuning procedure of the RandomForestClassifier model for Edge Server 4, which is another vital part of the federated learning framework. The task was to fine-tune hyperparameters and adjust model architectures to make them fit the particularities and restrictions of Edge Server 4. GridSearchCV method was implemented to systematically tune a prospective hyperparameter grid in search of the optimal configuration for the RandomForestClassifier algorithm.

While hyperparameter tuning of Edge Server 4 was carried out, the number of estimators (trees) 100, 200, and 300) maximum depth of trees (10,20, and 30), and minimum samples for splitting (4, 8, and 10) were systematically adjusted. This iterative approach helped to seek the balance between model complexity and generalization capability by considering the computational limitations and the data diversity specific to Edge Server 4.

Along with hyperparameter tuning, the training-test data split ratio has been appraised for Edge Server 4. The value of this ratio governs the amount of data allocated for modeling versus evaluation of unseen data which is further used to predict the performance of novel observations. To set the training-test ratio for Edge Server 4, an optimal balance between model training and evaluation was chosen. Also, the assignment of the weight to Edge Server 4 was vital for the federated learning setup. These weights represent how important or useful each edge server is for the model aggregation process. Weighting Edge Server 4 with an appropriate weight was aimed to maximize the intelligence of a distributed network by taking into account the quality of the data, the computational capabilities, and the distribution.

After optimizing the hyperparameters and weight allocation, the model at Edge Server 4 was designed to be highly efficient and transform the local data characteristics and computing resources into meaningful contributions to the collaborative model training process of the federated learning framework.

3.6.5 Global Model Configuration:

The process of model development and model updating at a federated learning environment consists of an ordered sequence of steps which includes collecting data from multiple distributed Edge Servers while keeping privacy and security over the data. Firstly, models are trained individually on every Edge Server via locally available data. Such models contain not only the knowledge that is relevant to this or that data set but also the specific features and distributions that are present in the various structures. When training, the parameters of the Edge models are encoded and then this information is transferred to the Central Server for the fusion. This process helps to create communication channels that act as data transfer pathways between the Edge servers and the Central server so that data is transferred securely and promptly.

Upon acquiring parameters via Edge Servers, the Central Server carries out weighted averaging for the combined representation from these parameters. The weighting system is based on certain things (data quality, model performance, and server reliability) and defines their importance level. This weighted aggregation approach thus provides the guarantee of appropriate evaluation of Edge Servers' contributions, thereby ensuring the integration of disparate sources of knowledge. Later,

these amalgamated parameters are used to update the global model and it captures all the intelligence that has been generated by all Edge Servers.

The weighted averaging can be processed as:

$$\theta = \sum_{i=1}^n \omega_i . P_i \quad (3)$$

(P_i) as the parameters received from the Edge Server.

(ω_i) as the weight assigned to the Edge Server.

(Θ) as the aggregated parameters for the global model.

(n) is the total number of Edge Servers.

This equation represents the weighted sum of parameters received from each Edge Server, where the weight assigned to each server reflects its contribution to the global model.

The newly revised global model is next uploaded onto the Central Server for either inferencing or further training. It works as an aggregate representing all distributed knowledge in all Edge Servers and, therefore, offers a better performance as well as improved reliability compared with individual models. The assembly and refinement of this global model is an iterative process that lets its parameters constantly adjust and improve following the changing data distributions and learning aims. Different feedback mechanisms may be used along the way to observe the performances of the overall model and to maintain the model's effectiveness and reliability in real-world applications.

Moreover, scalability and efficiency are fundamental considerations in the design of the federated learning system. Developing and perfecting the algorithms and protocols, the number of data that can be processed simultaneously increases and also the need for communication overhead lessens,

thus leading to a smooth addition of new Edge Servers and spanning across an intensive number of data. Besides, rigorous security and privacy precautions are sought during the parameter exchanging process and the model accumulation process to keep both sides' information. Encryption, authentication, and access control mechanisms are used to maintain privacy and maintain an unaltered state of data while building faith and classifying with regulatory standards. Taken into consideration, the global reasoning function is just a federated learning model of mixed knowledge from various sources which has been made possible by the application of privacy and security principles that prevail in machine learning.

Chapter 4

Result and Analysis

4.1 Performance Evaluation:

The performance of the model was done through an overall metric set to test its performance in making predictions. There were two disjoint sets corresponding to the training and testing subsets respectively which were used for both model training and performance evaluation to ensure an unbiased estimation of its performance. Along with the performance metrics such as accuracy, precision, recall, and f1-score were utilized to measure the model's performance. Such metrics together, impart a comprehensive insight into the model excellence, which in turn helps with informed decision-making and systematic improvements.

4.1.1 Accuracy

Accuracy functions as a cornerstone metric, indicating the overall model in making correct inferences. It determines the percentage of correctly classified cases of all evaluated instances. With a good accuracy score, acquired proof that the model is performing as expected, hence giving us the certainty of the model to make appropriate predictions. Ranging from data collection and preprocessing to model selection and evaluation, there are different challenges associated with the application of machine learning algorithms. The ratio of genuine positives and true negatives to the total number of positive and negative observations is used to compute it. Simply, accuracy represents the percentage of the machine learning model's total predictions that are accurate. The accuracy score calculation formula is:

$$Accuracy\ Score = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

True Positives (TP): These are the ones where the model has correctly predicted the positive class. TP means the number of times the model is correct in classifying positive instances. In the confusion matrix, for instance, TP is related to the diagonal elements, where the predicted value matches the real one for the positive class.

False Positives (FP): These are cases where the model incorrectly predicts a positive class as a negative one. FP is a variable for the number of instances that were not classified as positive by the algorithm. "FP" in the confusion matrix corresponds to the row values, which represent negative true labels (e.g., "0") but positive predicted labels (e.g., "1").

False Negatives (FN): These are the instances where the model is mislabeling the negative class when it is a positive one. FN in the code stands for an incorrect classification as negative by the model. In the confusion matrix, FN is the number in the columns whose true label is positive (such as "1") but the predicted label is negative (such as "0").

True Negatives (TN): These are the cases where the model correctly picks the negative class. TN stands for the rate of correct negative cases that are classified by the model. In a confusion matrix, TN is where the true label for the negative class matches the predicted label which constitutes the diagonal elements of the matrix.

4.1.2 Precision:

Precision is an important performance benchmark of classification algorithms, especially in the case of classifying true positives among other predicted classes. It determines the precision which is the ratio of correctly predicted positives to the number of positive instances the model picked

out. It can be said that mathematically precision is calculated as the quotient of real positives over the total number of true positives and false positives. TP

$$Precision\ Score = \frac{TP}{FP + TP} \quad (5)$$

In applications like medical diagnosis, spam detection, intrusion detection, etc. where misclassifications can have significant consequences, precision serves as a crucial measure of model reliability and effectiveness.

4.1.3 Recall:

The recall rate, which is also called sensitivity or true positive rate, is one of the essential metrics in the classification tasks and shows the model's ability to find all relevant examples. It determines the fraction of correct predictions made among all events that happen. Mathematically, recall is given by dividing true positives by the result of true positives plus false negatives.

$$Recall\ Score = \frac{TP}{FN + TP} \quad (6)$$

A model with a good recall score is one that well captures most of the positive cases, thus reducing the amount of false positives it produces. Recall in the field of medical screenings and fault detection is very important because the system can miss true-positive cases which can create critical issues.

4.1.4 F1-Score:

The F1 score is a measure that combines both precision and recall into a single value this reason it provides a balanced view of the model's performance. It is rather helpful in this case when the class distribution is uneven. The harmonic mean of precision and recall is computed as such, resulting in two metrics receiving the same consideration.

$$F1\ Score = 2. \frac{Precision * Recall}{Precision + Recall} \quad (7)$$

A precise F1 score indicates both high preciseness and high recall, hence the model gets a good accuracy in minimizing the false positives and the false negatives. Balancing both precision and recall is crucial in tasks like information retrieval or anomaly detection, where the F1 score is considered a comprehensive metric for checking model performance.

4.2 Result Analysis:

In this section, I analyze the performance of the secure and collaborative federated model based on the aforementioned evaluation metrics. I interpret the results to identify the strengths, and areas for improvement of the model.

4.2.1 Model Performance:

I trained all the edge servers locally and aggregated the server parameters to the global server to update the model. In Figure 4.1 the confusion matrix for the hyperparameter-tuned Edge server 1 is presented from this can calculate the accuracy, precision, recall, and F1 score.

If the servers are not balanced well as a result It can be seen that there are various issues with the NIDS. As the Infiltration label is far less than the other labels if I train the server based on the original imbalanced data, the infiltration detection is very poor and the whole NIDS becomes weak.

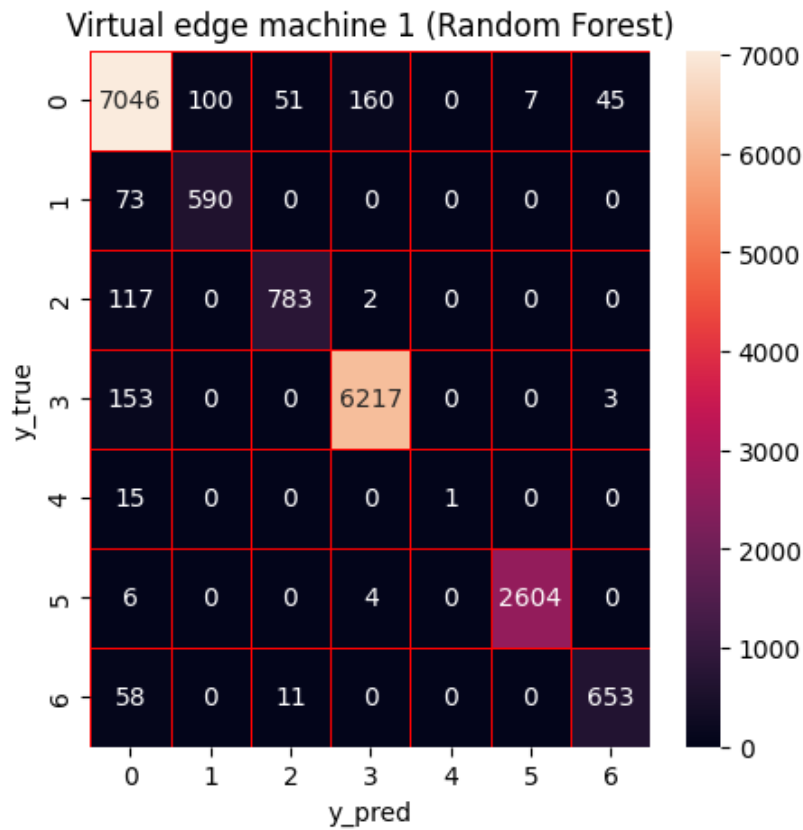


Figure 4.1 Confusion Matrix of Edge Server 1 with Imbalanced Dataset

Figure 4.1 shows how the confusion matrix of Edge server 1 if the server was trained of imbalanced data

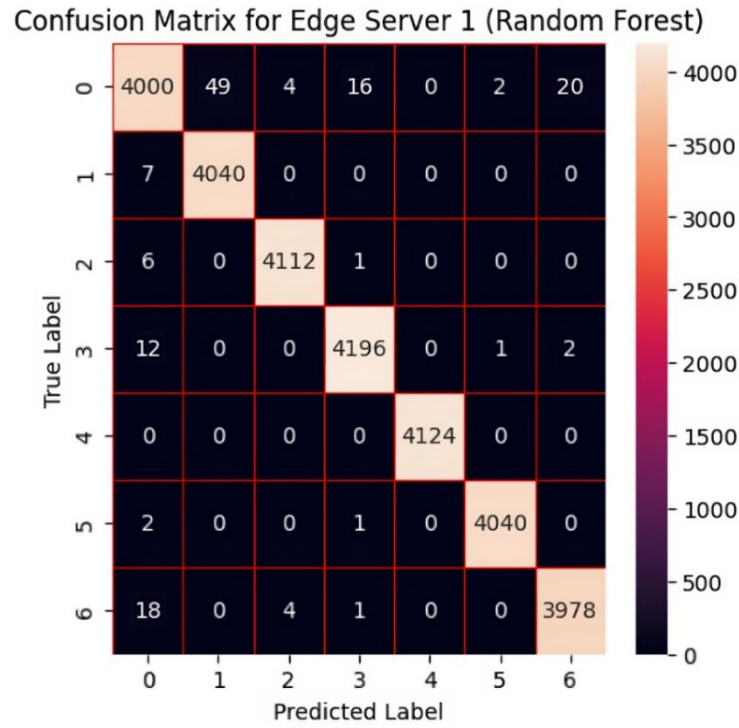


Figure 4.2 Confusion Matrix of Edge Server 1

The confusion matrix of the hyperparameter-tuned Edge Server 2 is presented in Figure 4.2

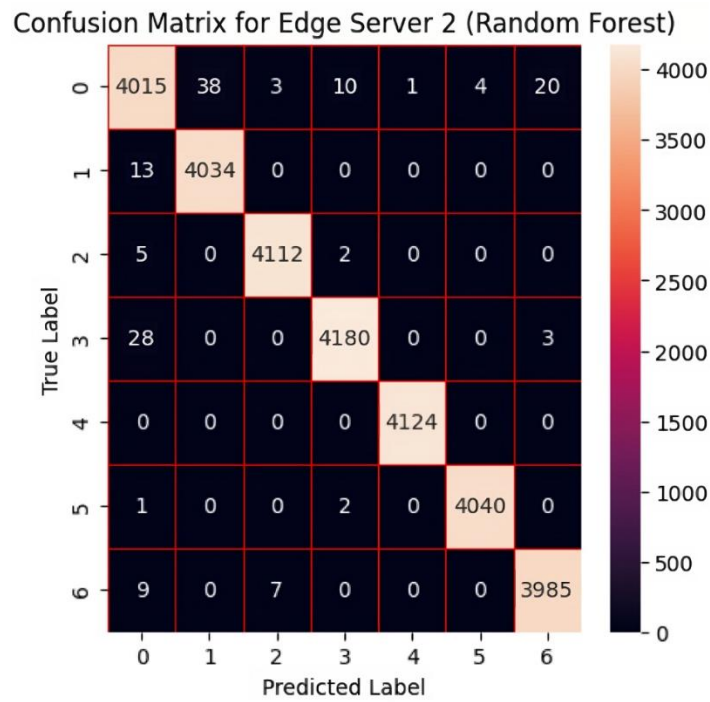


Figure 4.3 Confusion Matrix Edge Server 2

Figure 4.3 represents the confusion matrix for the hyperparameter-tuned Edge Server 3.

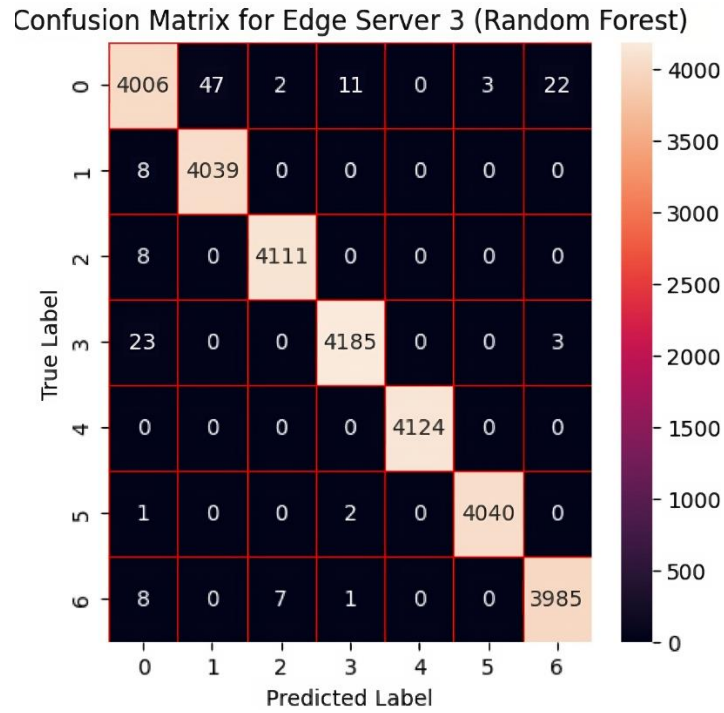


Figure 4.4 Confusion Matrix for Edge Server 3

Representation of hyperparameter-tuned Edge Server 4 is shown in Figure 4.4

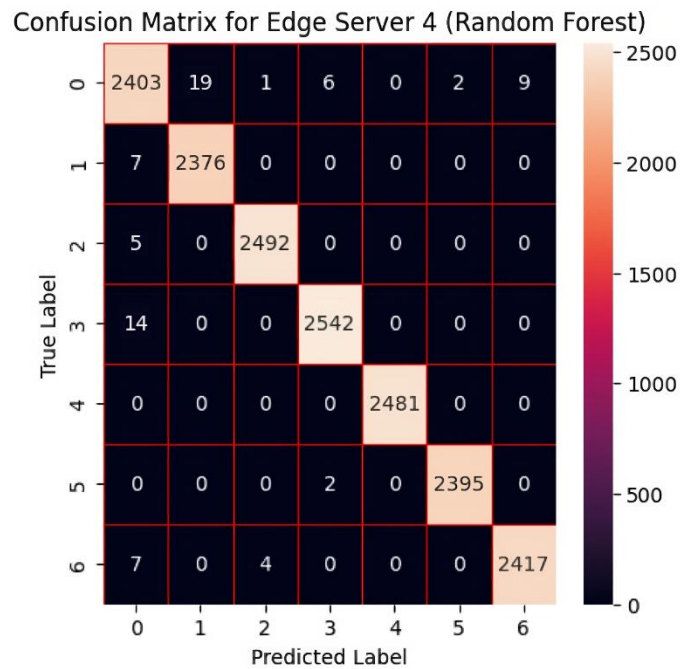


Figure 4.5 Confusion Matrix for Edge Server 4

The Confusion Matrix of the Global Server is presented in Figure 4.5

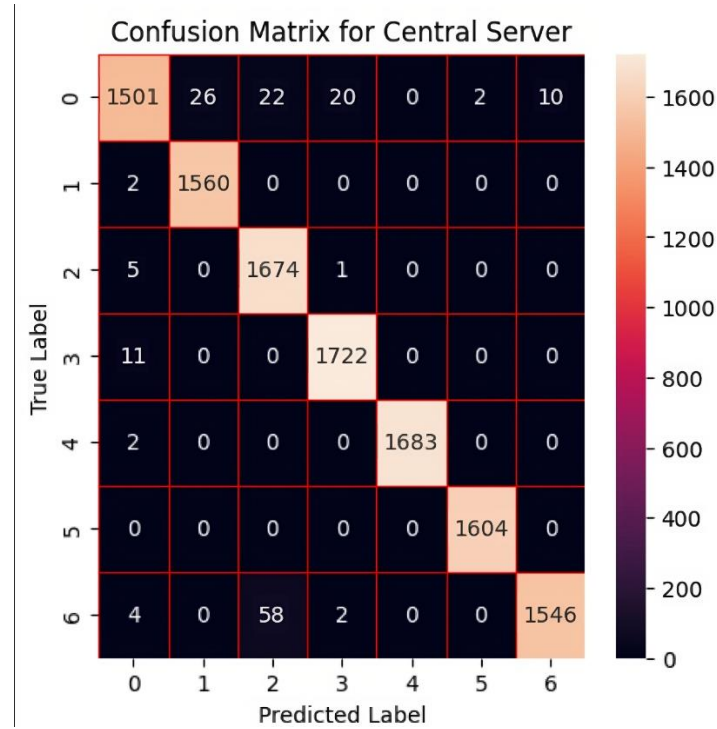


Figure 4.6 Confusion Matrix for Global Server

Figure 4.6 shows the overall performance of the servers.

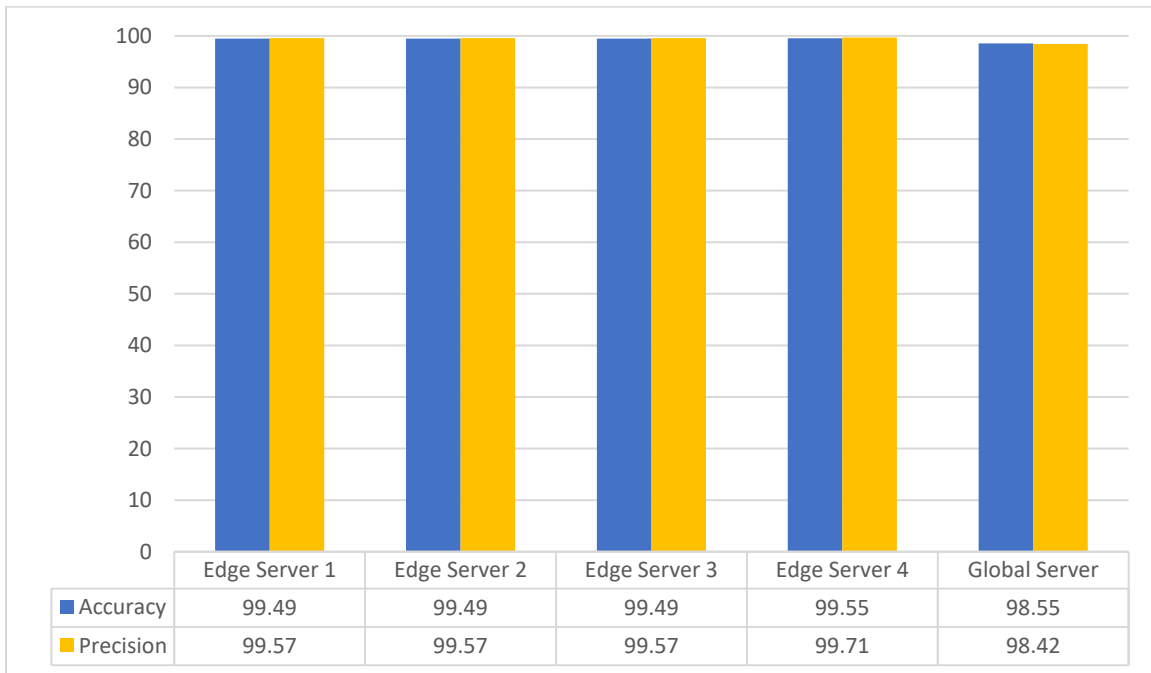


Figure 4.7 Model Performance

Figure 4.7 demonstrates the accuracy and precision of all the servers across the model.

The result of Edge Server 1, Edge Server 2, and Edge Server 3 is presented in Figure 4.1, Figure 4.2, and Figure 4.3. which has an accuracy of 99.49% and Edge Server 4 in Figure 4.4 has a 99.55% accuracy. The global server which gets updated by the parameters from the Edge servers has a 98.5% accuracy.

The precision, recall, and F1-score for detecting the malicious and nonmalicious activities of servers are shown in the tables.

Table 4.1 Test Results

	Accuracy (%)	Precision	Recall	F1-score
Edge Server 1	99.49	99.57	99.57	99.42
Edge Server 2	99.49	99.57	99.57	99.42
Edge Server 3	99.49	99.57	99.57	99.42
Edge Server 4	99.55	99.71	99.57	99.71
Global Server	98.5	98.42	98.57	98.14

Table 4.1 shows the server Accuracy, Precision, Recall, and F1- score of all the servers.

Table 4.2 Detection for Edge Server 1

	Precision	Recall	F1-score
BENIGN	0.99	0.98	0.98
Bot	0.99	1.0	0.99
BruteForce	1.0	1.0	1.0
Dos	1.0	1.0	1.0
Infiltration	1.0	1.0	1.0
PortScan	1.0	1.0	1.0
WebAttack	0.99	0.99	0.99

Tables 4.2 to 4.6 show the performance of all servers for detecting different kinds of attacks.

4.3 Detection for Edge Server 2

	Precision	Recall	F1-score
BENIGN	0.99	0.98	0.98
Bot	0.99	1.0	0.99
BruteForce	1.0	1.0	1.0
Dos	1.0	0.99	0.99
Infiltration	1.0	1.0	1.0
PortScan	1.0	1.0	1.0
WebAttack	0.99	1.0	1.0

Table 4.4 Detection for Edge Server 3

	Precision	Recall	F1-score
BENIGN	0.99	0.98	0.98
Bot	0.99	1.0	0.99
BruteForce	1.0	1.0	1.0
Dos	1.0	0.99	1.0
Infiltration	1.0	1.0	1.0
PortScan	1.0	1.0	1.0
WebAttack	0.99	1.0	0.99

Table 4.5 Detection for Edge Server 4

	Precision	Recall	F1-score
BENIGN	0.99	0.98	0.99
Bot	0.99	1.0	0.99
BruteForce	1.0	1.0	1.0
Dos	1.0	0.99	1.0
Infiltration	1.0	1.0	1.0
PortScan	1.0	1.0	1.0
WebAttack	1.0	0.99	1.0

Table 4.6 Detection for Global Server

	Precision	Recall	F1-score
BENIGN	0.98	0.95	0.97
Bot	0.98	1.0	0.99
BruteForce	0.95	1.0	0.97
Dos	0.99	0.99	0.99
Infiltration	1.0	1.0	1.0
PortScan	1.0	1.0	1.0
WebAttack	0.99	0.96	0.98

By comparing the performance results of the edge servers and the global server, it is possible to determine a significant difference between their efficiency. All three Edge Servers (Server 1, Server 2, and Server 3) demonstrate similar availability percentages throughout all time intervals, with percentages being 99.49%. Edge Server 4 deviated a little showing 99 as the initial uptime. Peaking at 99.55%. For precision, servers 1,2 and 3 have 99.57% but server 4 has 99.71%, and to ensure that the performance level stays above this mark over time. On the other hand, the graph for the global server is achieving 98. 5% accuracy and it has shown a 98.42% precision score.

Chapter 5

Conclusion and Future Work

Finally, I have seen that the federated way of network intrusion detection can be useful to boost network security. I have proven the practicality of a decentralized model training and aggregation technique by creating a federated environment with edge servers and a central server. The research reveals that learning models on local edge devices and then transmitting parameter data to the main server allows for intrusion detection that is robust while at the same time maintaining data privacy and lessening communication costs. On the other hand, the federated approach provides scaling and resiliency by transferring the computational operations to the edge devices. To be concise, my research extends and brings new ideas to the state of the art in the field of secure and collaborative network intrusion detection, which, in turn, helps to increase the security level of distributed systems.

Following this, future studies may be conducted on improving the federated method in network intrusion detection. For instance, the resources might be best allocated for the exploration of advanced privacy tools, like differential privacy, to make data privacy stronger in the federated setting. Also, research into adaptive models for dynamic updates and aggregation will be crucial for consideration for growing network environments. Additionally, the expansion of the federated framework towards heterogeneous edge devices could provide scalability and flexibility. With the help of these venues, I try to enhance the use of federated intrusion detection systems in various network types.

References:

- [1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, Jul. 2009, doi: 10.1145/1541880.1541882.
- [2] "Signature Based Intrusion Detection System Using SNORT," *ResearchGate*. [Online]. Available: https://www.researchgate.net/publication/274952404_Signature_Based_Intrusion_Detection_System_Using_SNORT
- [3] D. P. Gaikwad and R. C. Thool, "Intrusion Detection System Using Bagging with Partial Decision Tree-Base Classifier," *Procedia Computer Science*, vol. 49, pp. 92–98, 2015, doi: 10.1016/j.procs.2015.04.231.
- [4] M. H. Brendan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," *arXiv.org*, 2016. [Online]. Available: <https://arxiv.org/abs/1602.05629>
- [5] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated Learning: Strategies for Improving Communication Efficiency," *arXiv:1610.05492 [cs]*, Oct. 2017. [Online]. Available: <https://arxiv.org/abs/1610.05492>
- [6] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated Machine Learning," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, 2019, doi: 10.1145/3298981.

- [7] R. Sekar, Y. Guang, S. Verma, and T. Shanbhag, "A high-performance network intrusion detection system," in *Proceedings of the 6th ACM conference on Computer and communications security - CCS '99*, 1999, doi: 10.1145/319709.319712.
- [8] A. Javaid, Q. Niyaz, W. Sun, and M. Alam, "A Deep Learning Approach for Network Intrusion Detection System," in *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS)*, 2016, doi: 10.4108/eai.3-12-2015.2262516.
- [9] N. Farnaaz and M. A. Jabbar, "Random Forest Modeling for Network Intrusion Detection System," *Procedia Computer Science*, vol. 89, pp. 213–217, Jan. 2016, doi: 10.1016/j.procs.2016.06.047.
- [10] B. Brao and K. Swathi, Eds., "Fast kNN Classifiers for Network Intrusion Detection System," *ResearchGate*, Apr. 2017. [Online]. Available: https://www.researchgate.net/publication/317206932_Fast_kNN_Classifiers_for_Network_Intrusion_Detection_System
- [11] D. Perez, M. A. Astor, D. P. Abreu, and E. Scalise, "Intrusion detection in computer networks using hybrid machine learning techniques," in *2017 XLIII Latin American Computer Conference (CLEI)*, Cordoba, Argentina, 2017, pp. 1-10, doi: 10.1109/CLEI.2017.8226392.
- [12] A. Pattawaro and C. Polprasert, "Anomaly-Based Network Intrusion Detection System through Feature Selection and Hybrid Machine Learning Technique," in *2018 16th International Conference on ICT and Knowledge Engineering (ICT&KE)*, Bangkok, Thailand, 2018, pp. 1-6, doi: 10.1109/ICTKE.2018.8612331.

- [13] O. Faker and E. Dogdu, "Intrusion Detection Using Big Data and Deep Learning Techniques," in *2019 ACM Southeast Conference (ACMSE 2019)*, Kennesaw, GA, USA, Apr. 18-20, 2019, pp. 8, doi: 10.1145/3299815.3314439.
- [14] E. E. Abdallah, W. Eleisah, and A. F. Otoom, "Intrusion Detection Systems using Supervised Machine Learning Techniques: A survey," *Procedia Computer Science*, vol. 201, pp. 205–212, 2022, doi: 10.1016/j.procs.2022.03.029.
- [15] V. Rey, P. M. Sánchez Sánchez, A. Huertas Celdrán, and G. Bovet, "Federated learning for malware detection in IoT devices," *Computer Networks*, vol. 204, p. 108693, 2022. [Online]. Available: <https://doi.org/10.1016/j.comnet.2021.108693>
- [16] Y. Qin and M. Kondo, "Federated Learning-Based Network Intrusion Detection with a Feature Selection Approach," in *2021 International Conference on Electrical, Communication, and Computer Engineering (ICECCE)*, Kuala Lumpur, Malaysia, 2021, pp. 1-6, doi: 10.1109/ICECCE52056.2021.9514222.
- [17] J. Li, X. Tong, J. Liu, and L. Cheng, "An Efficient Federated Learning System for Network Intrusion Detection," *IEEE Systems Journal*, vol. 17, no. 2, pp. 2455-2464, Jun. 2023, doi: 10.1109/JSYST.2023.3236995.
- [18] R. Panigrahi and S. Borah, "A detailed analysis of CICIDS2017 dataset for designing Intrusion Detection Systems," *International Journal of Engineering & Technology*, vol. 7, pp. 479-482, 2018.

[19] J. K. Jaiswal and R. Samikannu, "Application of Random Forest Algorithm on Feature Subset Selection and Classification and Regression," in *2017 World Congress on Computing and Communication Technologies (WCCCT)*, Tiruchirappalli, India, 2017, pp. 65-68, doi: 10.1109/WCCCT.2016.25.

[20] T. Puślecki and K. Walkowiak, "Hyperparameters Optimization Using GridSearchCV Method for TinyML Models," in *Progress on Pattern Classification, Image Processing and Communications. CORES IP&C 2023 2023. Lecture Notes in Networks and Systems*, vol. 766, R. Burduk, M. Choraś, R. Kozik, P. Ksieniewicz, T. Marciniak, and P. Trajdos, Eds. Springer, Cham, 2023. doi: 10.1007/978-3-031-41630-9_7